

SECAUDIT 050 - CMS Architecture Assessment: Direct-Access Hardening, and an Honest Accounting

SECURITY AUDIT / PUBLIC REMEDIATION RECORD

Date: August 21, 2026 **Status:** Fixed item closed (code shipped, live verification pending) - architecture review dispositioned

SECAUDIT 050 set out to do something the earlier audits had not: step back from line-by-line code review and assess the whole content-management system against the established patterns that mature publishing, media-library, database, security, recovery, and federation systems follow. It was a structural review, not a hunt for a single bug. It produced one security fix worth disclosing now, one data-integrity verification, and - more importantly - an honest accounting of a foundational problem that earlier reviews should have caught and did not.

The security finding that is fixed. SnapSmack's protection for sensitive server-side files was a hand-maintained, Apache-only deny-list, and it had drifted out of step with the code it was meant to protect. Several core PHP includes - the updater, the release-signing public key, the skin registry, the shared-asset manifest inventory, and the real authentication include - could be requested directly rather than only through the application, and the deny-list still named files that no longer existed while omitting the authentication file that mattered most. On any server that did not enforce the Apache rules, or through the gaps in a list that had fallen behind, those endpoints were reachable on their own. Release 0.7.544 closes this: each of those includes now carries a server-agnostic guard that refuses a direct request regardless of web server, and the Apache deny-list was reconciled to the files that actually ship. The code fix is released; confirming it on every live installation happens through the normal update-and-repair path and is recorded as pending rather than quietly marked done.

The foundational problem, stated plainly because it is embarrassing and belongs on the record. This architecture review exists in large part because an earlier audit examined the code, found it clean - correct database parameters, escaping, tokens, authentication, signed updates - and pronounced the system sound. It missed that the system had no single, enforced definition of what a published item even is. Legacy solo publishing stored a photograph as a bare image row, not as a post; newer group and longform features used a separate post record. That meant publication state, dates, ownership, engagement, collections, and federation identity had no one home and could take on competing meanings. Clean code sitting on an undefined content model passes a code review and still fails an architecture review. The first review asked "is the code correct?" when it also needed to ask "is the fundamental unit of content even defined?" - and did not. We are publishing that miss for the same reason we published the audit-process correction in SECAUDIT 046: a foundational flaw that a review waved through belongs in the security record, not tidied away. The remediation of the content model itself (releases 0.7.534D onward) is real work in progress and is not being represented as finished.

The data-integrity verification, including a mistake made during it. A long-open check for orphaned community records - comments, likes, and reactions with no valid parent - was completed against the highest-risk site's data and confirmed zero truly orphaned records. That result comes with an honest caveat. An earlier version of the check used a posts-only definition of "orphan," which is wrong for how this system actually stores engagement: comments and likes are keyed to the image identifier by design, and the display reads them that way. That wrong assumption briefly removed legitimate community comments during analysis; they were restored from backup, and the corrected, display-aware check then confirmed there was no orphan problem to begin with. The lesson is recorded: an integrity check must match how a table is actually read, not how one assumes it should be.

What remains, and why it is not detailed here. The review dispositioned eight findings. Two are closed - the direct-access hardening above and the data-integrity verification. The rest are deferred with named owners and reasons, and they include further hardening of administrator authorization and of URL-to-handler routing. Those remaining items are tracked internally and are deliberately not described in detail in this public record, because publishing a map of weaknesses that are not yet fixed would itself create risk. Each will be disclosed the way this one is - concretely, and only

once it is closed. This report covers current development code; per-installation and fleet-wide verification of the shipped fix is the next step and is recorded as outstanding, not complete.